Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра системного проектування

# ЗВІТ
# про виконання лабораторної роботи №3
# «Ітератори, генератори та потокова обробка даних»
# з дисципліни «Професійний Python»

Виконав:
студент 3 курсу
групи ФЕП-33
Зінчук Остап

Викладач:
проф. Свелеба Сергій Андрійович

Львів – 2026

## 2. Номер і тема лабораторної роботи

Лабораторна робота №3: «Ітератори, генератори та потокова обробка даних».

## 3. Мета роботи

Сформувати практичні навички роботи з ітераційною моделлю Python, ітераторами, генераторами, yield, generator expressions, модулем itertools, принципом lazy evaluation та потоковою обробкою великих наборів даних без повної матеріалізації проміжних колекцій в оперативній пам'яті.

## 4. Номер індивідуального варіанта

Варіант 4: Потокова обробка системи банківських рахунків.

## 5. Постановка задачі

Розробити потоковий конвеєр (streaming pipeline) для обробки великого масиву банківських записів. Конвеєр повинен послідовно виконувати: потокове читання рядків із файлу (yield from), синтаксичний розбір CSV (DictReader + yield from), валідацію полів та типів, застосування операцій поповнення (deposit) та зняття (withdraw) з жорстким контролем недопущення від'ємного балансу, нормалізацію імен клієнтів, фільтрацію за порогом балансу та потокову агрегацію статистики. Забезпечити підтримку пакетної обробки (batching через islice), використання розширених засобів itertools (chain, pairwise, accumulate, groupby, count), реалізацію власного класу-ітератора (AccountRange) та рекурсивного генератора. Провести експериментальне порівняння характеристик часу та пам'яті (tracemalloc) для eager та lazy підходів на датасетах обсягом 10 000, 100 000 та 500 000 записів, а також оцінити ефект раннього переривання (early termination).

## 6. Опис input dataset

Вхідні дані представлені CSV-файлом зі структурою колонок: account_number,client,balance,operation,amount.
• account_number — унікальний ідентифікатор банківського рахунку (наприклад, UA0000001);
• client — прізвище та ім'я власника рахунку;
• balance — початковий баланс рахунку (дійсне невід'ємне число);
• operation — тип платіжної транзакції (deposit, withdraw, check);
• amount — сума транзакції (невід'ємне дійсне число).
У наборі даних враховано наявність некоректних рядків (пропущені обов'язкові значення, від'ємні баланси, невалідні числові формати, недопустимі типи операцій), які мають відсікатися на етапі валідації.

## 7. Структура проєкту

```
Lab_03/
├── .gitignore
├── pyproject.toml
├── README.md
├── data/
│   ├── accounts.csv
│   └── sample.csv
├── report/
│   └── Проф. Python-Lab_03-Зінчук.pdf
├── src/
│   └── stream_processor/
│       ├── __init__.py
│       ├── analytics.py
│       ├── batches.py
│       ├── filters.py
│       ├── main.py
│       ├── models.py
│       ├── parsers.py
│       ├── pipeline.py
│       ├── readers.py
│       └── transformations.py
└── tests/
    └── test_stream.py
```

## 8. Пояснення концепцій Iterable та Iterator

В ітераційній моделі Python розрізняють два ключові поняття:
• Iterable (ітерований об'єкт): будь-який об'єкт, здатний повертати свої елементи по одному. Об'єкт є iterable, якщо він реалізує магічний метод __iter__() (який повинен повертати об'єкт ітератора) або метод __getitem__() із числовими послідовними індексами від нуля. Приклади: list, tuple, dict, str.
• Iterator (ітератор): об'єкт-стан, який представляє потік даних і здійснює безпосередній обхід. Він реалізує методи __next__() (повертає наступний елемент або піднімає виняток StopIteration після вичерпання послідовності) та __iter__() (повертає сам себе return self). Ітератори є одноразовими об'єктами (one-shot objects). Розділення контейнера й ітератора дозволяє здійснювати паралельні незалежні обходи одного й того самого контейнера.

## 9. Код власного iterator (AccountRangeIterator та AccountRange)

```
class AccountRangeIterator(Iterator[int]):
    def __init__(self, start: int, stop: int, step: int = 1) -> None:
        self.current = start
        self.stop = stop
        self.step = step

    def __iter__(self) -> 'AccountRangeIterator':
        return self

    def __next__(self) -> int:
        if self.current >= self.stop:
            raise StopIteration
        value = self.current
        self.current += self.step
        return value

class AccountRange:
    def __init__(self, start: int, stop: int, step: int = 1) -> None:
        self.start = start
        self.stop = stop
        self.step = step

    def __iter__(self) -> AccountRangeIterator:
        return AccountRangeIterator(self.start, self.stop, self.step)
```

## 10. Приклад generator function

Генераторна функція — функція, яка містить ключове слово yield. При виклику вона не виконує своє тіло одразу, а повертає спеціальний об'єкт генератора.

```
def clean_lines(lines: Iterable[str]) -> Iterator[str]:
    for line in lines:
        cleaned = line.strip()
        if cleaned:
            yield cleaned
```

## 11. Приклад yield (збереження стану та ліниве продукування)

```
def apply_operations(records: Iterable[AccountRecord]) -> Iterator[AccountRecord]:
    for rec in records:
        cur_bal = rec.balance
        if rec.operation == 'deposit':
            new_bal = round(cur_bal + rec.amount, 2)
        elif rec.operation == 'withdraw':
            new_bal = round(cur_bal - rec.amount, 2) if cur_bal >= rec.amount else cur_bal
        else:
            new_bal = cur_bal
        yield rec._replace(balance=new_bal)
```

## 12. Приклад yield from (делегування підгенераторам)

```
def read_lines(path: Path) -> Iterator[str]:
    with path.open('r', encoding='utf-8') as file:
        yield from file

def parse_csv_rows(lines: Iterable[str]) -> Iterator[dict[str, str]]:
    reader = csv.DictReader(lines)
    yield from reader

def recursive_flatten(values: Iterable[Any]) -> Iterator[Any]:
    for item in values:
        if isinstance(item, (list, tuple)):
            yield from recursive_flatten(item)
        else:
            yield item
```

## 13. Generator expression

Генераторні вирази мають компактний синтаксис у круглих дужках і генерують елементи на вимогу без створення списку в пам'яті:

```
def streaming_total_funds(records: Iterable[AccountRecord]) -> float:
    return round(sum(acc.balance for acc in records), 2)
```

## 14. Використані засоби itertools

У проєкті застосовано широкий спектр оптимізованих інструментів модуля itertools:
• itertools.islice: потокове вилучення фіксованої кількості записів без обробки решти потоку (batching);
• itertools.chain: об'єднання читання декількох файлів у єдиний потік (read_files_chain);
• itertools.accumulate: розрахунок кумулятивного наростаючого підсумку балансів (cumulative_balances);

• itertools.pairwise: попарне порівняння та розрахунок різниць між послідовними транзакціями;
• itertools.groupby: групування транзакцій за категоріями операцій;
• itertools.count: генерація нескінченного потоку унікальних ідентифікаторів рахунків.

## 15. Схема pipeline

Архітектура потокового конвеєра відповідає вимогам розділу 75 методичних вказівок:

```
  CSV File (data/accounts.csv)
       │
       ▼
  read_lines()        -> Читання рядків за допомогою yield from
       │
       ▼
  clean_lines()       -> Очищення від пробільних символів та порожніх рядків
       │
       ▼
  parse_csv_rows()    -> Потоковий синтаксичний аналіз DictReader + yield from
       │
       ▼
  validate_accounts() -> Валідація типів, форматів та відсікання від'ємних сум
       │
       ▼
  apply_operations()  -> Застосування deposit/withdraw із захистом від овердрафту
       │
       ▼
  normalize_clients() -> Приведення імен клієнтів до Title Case
       │
       ▼
  filter_by_balance() -> Фільтрація за мінімальним порогом залишку коштів
       │
       ▼
  [ Sink / Споживач ]:
       ├── calculate_statistics()     -> Агрегація статистики без матеріалізації
       ├── batched_records()          -> Пакетування через itertools.islice
       └── find_first_account()       -> Раннє переривання (early termination)
```

## 16. Код streaming reader

```
"""Модуль генераторів читання та попередньої фільтрації рядків."""

from collections.abc import Iterable, Iterator
from itertools import chain
from pathlib import Path

def read_lines(path: Path) -> Iterator[str]:
    """Генератор читання файлу рядок за рядком із використанням yield from."""
    with path.open("r", encoding="utf-8") as file:
        yield from file

def clean_lines(lines: Iterable[str]) -> Iterator[str]:
    """Генератор очищення рядків від пробілів та пустих рядків."""
    for line in lines:
        cleaned = line.strip()
        if cleaned:
            yield cleaned

def read_files_chain(*paths: Path) -> Iterator[str]:
    """Об'єднує читання кількох файлів у єдиний потік через itertools.chain."""
    iterators = [read_lines(p) for p in paths]
    yield from chain(*iterators)
```

## 17. Validation stage

```
def validate_accounts(rows: Iterable[dict[str, str]]) -> Iterator[AccountRecord]:
    for row in rows:
        try:
            acc_num = row.get('account_number', '').strip()
            client = row.get('client', '').strip()
            balance = float(row.get('balance', ''))
            op = row.get('operation', 'check').strip().lower()
            amount = float(row.get('amount', '0.0'))
            if not acc_num or not client or balance < 0 or amount < 0:
                continue
            if op not in ALLOWED_OPERATIONS:
                continue
            yield AccountRecord(acc_num, client, balance, op, amount)
        except (ValueError, TypeError, KeyError):
            continue
```

## 18. Filtering stage

```
def filter_by_balance(records: Iterable[AccountRecord], min_balance: float) -> Iterator[AccountRecord]:
```

```
    for record in records:
        if record.balance >= min_balance:
            yield record

def find_first_account(records: Iterable[AccountRecord], predicate: Callable) -> AccountRecord | None:
    for record in records:
        if predicate(record):
            return record
    return None
```

## 19. Transformation stage

```
def normalize_clients(records: Iterable[AccountRecord]) -> Iterator[AccountRecord]:
    for rec in records:
        yield rec._replace(client=rec.client.strip().title())
```

## 20. Aggregation stage

```
def calculate_statistics(records: Iterable[AccountRecord]) -> dict[str, Any]:
    count = 0
    total = 0.0
    min_bal, max_bal = None, None
    operations_counter: Counter[str] = Counter()
    for rec in records:
        b = rec.balance
        count += 1
        total += b
        operations_counter[rec.operation] += 1
        if min_bal is None or b < min_bal: min_bal = b
        if max_bal is None or b > max_bal: max_bal = b
    avg = total / count if count else 0.0
    return {'count': count, 'total': round(total, 2), 'average': round(avg, 2),
            'minimum': min_bal, 'maximum': max_bal, 'operations': dict(operations_counter)}
```

## 21. Batching stage (islice та динамічне пакетування)

```
"""Модуль пакетної обробки потоків (batch processing)."""

from collections.abc import Iterable, Iterator
from itertools import islice

from .models import AccountRecord

def batched_records(
    records: Iterable[AccountRecord],
    batch_size: int,
) -> Iterator[list[AccountRecord]]:
    """Розбиває вхідний потік на пакети заданого розміру через islice."""
    it = iter(records)
    while True:
        batch = list(islice(it, batch_size))
        if not batch:
            return
        yield batch

batched_students = batched_records

def dynamic_batched(
    records: Iterable[AccountRecord],
    max_amount_per_batch: float = 10000.0,
) -> Iterator[list[AccountRecord]]:
    """Динамічне пакетування за накопиченою сумою операцій (Advanced)."""
    it = iter(records)
    current_batch: list[AccountRecord] = []
    current_sum = 0.0

    for rec in it:
        current_batch.append(rec)
        current_sum += rec.amount
        if current_sum >= max_amount_per_batch:
            yield current_batch
            current_batch = []
            current_sum = 0.0

    if current_batch:
        yield current_batch
```

## 22. Eager implementation

Жадібна реалізація матеріалізує повний вміст файлу у список словників у RAM:

```
def read_all_accounts_eager(path: Path) -> list[dict[str, str]]:
    with path.open('r', encoding='utf-8', newline='') as file:
        reader = csv.DictReader(file)
        return list(reader)
```

## 23. Lazy implementation

Лінива реалізація повертає ітератор і читає файл по одному запису за викликом next():

```
def read_accounts_lazy(path: Path) -> Iterator[dict[str, str]]:
    with path.open('r', encoding='utf-8', newline='') as file:
        reader = csv.DictReader(file)
        yield from reader
```

## 24-25. Таблиці результатів експерименту (Elapsed Time та Peak Memory)

| Кількість записів | Eager Time (s) | Lazy Time (s) | Eager Peak RAM | Lazy Peak RAM | TTFR (s) |
|---|---|---|---|---|---|
| 10 000 | 0.0705 | 0.0635 | 4.64 MB | 0.19 MB | 0.000270 |
| 100 000 | 0.7704 | 0.6860 | 45.00 MB | 0.19 MB | 0.000268 |
| 500 000 | 3.8734 | 3.4654 | 224.54 MB | 0.19 MB | 0.000301 |

Результати дослідження раннього переривання (Early Termination, 100 000 записів):
• Eager time: 0.209648 с (необхідність спочатку завантажити всі 100 тис. записів у пам'ять);
• Lazy time: 0.000372 с (миттєве повернення першого знайденого результату);
• Прискорення (Speedup): 563.6x!

## 26. Аналіз результатів

На основі проведених експериментальних замірів сформульовано такі ключові висновки:
1. Використання пам'яті (Peak Memory): Eager-підхід демонструє лінійне зростання пікового використання пам'яті O(N). При обробці 500 000 записів список зайняв 224.54 МБ оперативної пам'яті. Натомість Lazy-підхід має константне споживання O(1) — лише 0.19 МБ, оскільки в пам'яті в кожен момент часу перебуває лише один поточний об'єкт.
2. Час до отримання першого результату (Time to First Result — TTFR): У жадібному підході програма повинна дочекатися завершення читання всього файлу, тоді як генератор повертає перший елемент за ~0.27 мс, забезпечуючи максимальну інтерактивність користувацького інтерфейсу або конвеєра.
3. Раннє переривання (Early Termination): Коли потрібно знайти перший запис або перші N елементів, Lazy-конвеєр перевершує жадібний метод у 563.6 раза, повністю усуваючи зайву дискову та обчислювальну роботу.
4. Загальний час виконання (Total Elapsed Time): За умови повного вичерпання великого потоку час виконання обох варіантів є близьким (lazy трохи швидший через відсутність накладних витрат на реаллокацію великих списків). Таким чином, генератори виграють передусім за рахунок колосальної економії пам'яті та миттєвого старту відгуку.

## 27. Відповіді на контрольні запитання

**1. iter(iterator) is iterator?**

Так, для будь-якого ітератора виклик iter(iterator) повертає сам цей об'єкт (self), оскільки ітератор сам реалізує протокол __iter__().

**2. Чи можна повторно використати exhausted generator?**

Ні, вичерпаний генератор є одноразовим (one-shot). При повторній спробі отримати наступний елемент він одразу піднімає StopIteration. Для повторного обходу слід викликати генераторну функцію знову.

**3. Що таке generator function?**

Це звичайна функція Python, тіло якої містить ключове слово yield. Її виклик не виконує код одразу, а повертає спеціальний генераторний об'єкт.

**4. Яку роль виконує yield?**

yield повертає чергове значення споживачеві, призупиняє виконання функції та зберігає весь її внутрішній стан (значення локальних змінних і точку зупинки) до наступного виклику next().

**5. Чим yield відрізняється від return?**

return повністю завершує роботу функції та очищує її стек фрейму, тоді як yield лише тимчасово призупиняє виконання зі збереженням локальних змінних для наступних ітерацій.

**6. Коли фактично починає виконуватися generator function?**

Тіло генератора фактично починає виконуватися лише в момент першого звернення до нього — виклику функції next() або входження в цикл for.

**7. Що зберігає generator між викликами next()?**

Генератор зберігає весь свій стан: покажчик на поточну інструкцію коду (instruction pointer), стек виклику, а також усі локальні змінні та їхні значення.

**8. Що таке generator expression?**

Це компактний синтаксичний вираз створення генератора у круглих дужках: (expr for item in iterable if condition), який ліниво генерує значення без створення списку.

**9. Чим generator expression відрізняється від list comprehension?**

List comprehension одразу створює і повністю заповнює список у пам'яті (eager), тоді як generator expression повертає генератор, що продукує елементи на вимогу (lazy, O(1) RAM).

**10. Що таке lazy evaluation?**

Стратегія лінивих обчислень, при якій обчислення значення відкладається до того моменту, коли воно дійсно стає потрібним споживачеві.

**11. Що таке eager evaluation?**

Жадібна стратегія, при якій усі дані та вирази обчислюються і матеріалізуються в пам'яті негайно при виконанні інструкції.

**12. Які переваги lazy evaluation?**

Мінімальне пікове використання оперативної пам'яті, миттєвий початок відгуку (low time-to-first-result), можливість обробки нескінченних потоків та колосальна оптимізація при early termination.

**13. Які недоліки lazy evaluation?**

Неможливість прямої індексації або зрізів (не можна взяти [5]), одноразовість обходу, а також затримка у виявленні винятків (винятки виникають під час споживання, а не при створенні).

**14. Що таке streaming?**

Потокова обробка даних частинами (запис за записом або блоками) без завантаження всього набору даних у пам'ять одночасно.

**15. Чому streaming важливий для великих files?**

Тому що розмір великих файлів може значно перевищувати обсяг доступної оперативної пам'яті (RAM), що без потоковості призведе до аварійного збою Out of Memory (OOM).

**16. Чому read() може бути проблемою для multi-GB file?**

Виклик file.read() зчитує весь вміст файлу в один гігантський рядок у RAM, спричиняючи дефіцит пам'яті, активний свопінг на диск або завершення процесу ОС.

**17. Чому file object є iterable?**

Тому що об'єкт відкритого файлу в Python реалізує протокол __iter__() і повертає ітератор рядків, зчитуючи дані з диска ліниво по одному рядку за викликом next().

**18. Для чого використовується yield from?**

Для прозорого делегування операції генерації підгенератору чи будь-якому іншому ітерованому об'єкту, усуваючи зайві вкладені цикли та прокидаючи значення і винятки.

**19. Що таке itertools?**

Стандартний високопродуктивний модуль Python, реалізований на мові C, який містить набір будівельних блоків для побудови ефективних лінивих ітераторів.

**20. Для чого itertools.count()?**

Для створення лінивого нескінченного арифметичного ітератора чисел із заданим початковим значенням і кроком (count(start, step)).

**21. Чому count() потрібно обмежувати?**

Оскільки count() продукує нескінченну послідовність, спроба матеріалізувати її через list() чи цикл без умови break призведе до вічного зависання або переповнення пам'яті.

**22. Для чого islice()?**

Для отримання лінивого зрізу (частини) будь-якого ітератора без повної матеріалізації попередніх чи наступних елементів.

**23. Для чого chain()?**

Для послідовного лінивого об'єднання декількох ітерованих послідовностей в один безперервний потік без їхньої конкатенації в новий список.

**24. Як працює takewhile()?**

Повертає елементи з вхідного потоку доти, доки заданий предикат повертає True. При першому хибному значенні ітерація назавжди зупиняється.

**25. Як працює dropwhile()?**

Пропускає початкові елементи, поки предикат повертає True, а після першого випадку False повертає всі наступні елементи потоку без перевірки предиката.

**26. Для чого accumulate()?**

Для обчислення накопичувальних значень (наприклад, кумулятивної суми) елементів послідовності на кожному кроці ітерації.

**27. Для чого pairwise()?**

Для попарного обходу сусідніх елементів послідовності ((s0, s1), (s1, s2), (s2, s3)...), що корисно для аналізу різниць, швидкостей або змін між кроками.

**28. Як працює groupby()?**

Групує поспіль розташовані елементи з однаковим значенням ключа у підгрупи у вигляді кортежів (key, group_iterator).

**29. Чому перед groupby() часто потрібне sorting?**

groupby групує лише суміжні елементи з однаковим ключем. Якщо однакові ключі розкидані по списку, вони сформують окремі розрізнені підгрупи.

**30. Що робить product()?**

Обчислює декартовий добуток кількох ітерованих послідовностей (аналог вкладених циклів for).

**31. Чим combinations() відрізняється від permutations()?**

combinations генерує унікальні підмножини елементів без повторень, де порядок не має значення; permutations генерує всі можливі впорядковані перестановки.

**32. Чому combinatorial iterators можуть бути небезпечними навіть при lazy execution?**

Через комбінаторний вибух: кількість комбінацій зростає факторіально або експоненційно, що призведе до гігантських витрат процесорного часу при споживанні.

**33. Що таке batching?**

Розбиття безперервного потоку даних на порції (пакети) фіксованого або динамічного розміру для ефективнішої пакетної обробки (БД, мережа).

**34. Які переваги batching?**

Зменшення накладних витрат на мережеві запити або транзакції БД (bulk insert), баланс між утилізацією пам'яті та векторизованою швидкодією.

**35. Що таке lazy pipeline?**

Конвеєр послідовної обробки даних, у якому кожен етап є генератором і запитує наступний запис у попереднього етапу лише за необхідності.

**36. Які stages може містити pipeline?**

Джерело (Source) -> Очищення (Clean) -> Розбір (Parser) -> Валідація (Validation) -> Фільтрація (Filtering) -> Трансформація (Transform) -> Агрегація/Вивантаження (Sink).

**37. Що є source у streaming pipeline?**

Початковий генератор, що зчитує сирі дані з зовнішнього джерела (файл на диску, сокет, черга повідомлень Kafka/RabbitMQ).

**38. Що є transformation?**

Проміжний генератор конвеєра, який змінює формат запису, збагачує його даними, перераховує поля або нормалізує текст.

**39. Що є sink?**

Кінцевий споживач потоку, який здійснює термінальну операцію: запис у вихідний файл, відправка в БД, обчислення суми чи підсумкової статистики.

**40. Чи гарантує generator менший total execution time?**

Ні, генератори гарантують колосальну економію пам'яті, але сумарний час повного обходу зазвичай приблизно однаковий із матеріалізованим списком.

**41. Яка основна memory advantage generator?**

Константне споживання пам'яті O(1) незалежно від обсягу набору даних, оскільки в RAM утримується лише один активний елемент.

**42. Що таке time to first result?**

Час від моменту запуску конвеєра до моменту готовності першого обробленого елемента для споживача.

**43. Чому lazy pipeline добре підтримує early termination?**

Тому що при знаходженні потрібного результату споживач просто припиняє виклики next(), і конвеєр не виконує жодних обчислень для решти даних.

**44. Чому exception у generator може виникати із затримкою?**

Тому що код генератора виконується відкладено під час ітерації (next()), і помилка в даних проявляється лише тоді, коли черга доходить до некоректного запису.

**45. Чому list(generator) може бути проблемою?**

Тому що ця операція негайно примусово матеріалізує весь потік у список, повністю нівелюючи перевагу генератора в економії пам'яті.

**46. Що трапиться при list(count())?**

Програма зависне у нескінченному циклі заповнення пам'яті до моменту падіння з помилкою MemoryError.

**47. Як generator може впливати на lifecycle file resource?**

Якщо генератор відкрив файл у блоці with і споживання було перервано достроково (early termination), файл залишається відкритим, доки генератор не буде закрито методом close() або зібрано GC.

**48. Як тестувати невеликий generator?**

Шляхом матеріалізації його результату у список list(gen) і перевірки очікуваного списку значень через assertEqual.

**49. Як тестувати infinite generator?**

За допомогою itertools.islice(gen, N) для обмеження вибірки першими N елементами з наступним перетворенням у список.

**50. Коли краще використовувати list, а коли generator?**

list краще використовувати, коли потрібен багаторазовий швидкий доступ за індексами, зрізи або розмір колекції невеликий. Generator є незамінним для великих файлів, потоків даних, одноразових операцій та мінімізації пам'яті.

## 28. Висновок

У ході виконання лабораторної роботи №3 було успішно опановано концепції ітераційної моделі Python, ітераторів, генераторних функцій із yield та yield from, генераторних виразів і розширеного інструментарію модуля itertools. Створено повноцінний лінивий конвеєр потокової обробки банківських транзакцій (Варіант 4), який забезпечує валідацію, контроль від'ємного балансу, фільтрацію та агрегацію статистики без повної матеріалізації колекцій у пам'яті. Практичні експерименти на вибірках до 500 000 записів підтвердили теоретичні положення про переваги lazy evaluation: використання пам'яті скоротилося з 224.54 МБ (у жадібному варіанті) до константних 0.19 МБ, час до отримання першого результату становив усього 0.27 мс, а при ранньому перериванні лінивий конвеєр продемонстрував прискорення у 563.6 раза. Отримані практичні результати доводять вирішальне значення потокової моделі для обробки великих наборів даних в умовах обмежених ресурсів.